

操作系统，2026

第11讲 设备管理(1)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

设备管理概述

02

I/O硬件组成原理

03

I/O软件技术

04

I/O软件层次

设备管理概述

- 控制计算机的所有I/O设备是操作系统的主要功能之一
- I/O子系统是操作系统中负责管理输入输出设备的部分，它负责控制所有输入输出设备以完成期望的数据传送，实现操作系统的设备管理功能
- 在设备和系统的其他部分之间提供简单且易于使用的接口，并且该接口对于所有设备应该是相同的——设备无关性

设备管理的特点

- CPU与I/O设备的速度差别大，I/O的性能经常成为系统性能的瓶颈
- 操作系统庞大复杂的原因之一——资源多、杂，并发，均来自I/O
- 理解I/O的工作过程与结构是理解操作系统的工作过程与结构的关键
- 与其他功能联系密切，特别是文件系统

设备管理概述

- I/O设备在速度上覆盖了巨大的范围，要使软件在跨越这么多数量级的数据率下保证性能优良，给操作系统的I/O管理造成了相当大的压力

Device	Data rate
Keyboard	10 bytes/sec
Mouse	100 bytes/sec
56K modem	7 KB/sec
Telephone channel	8 KB/sec
Dual ISDN lines	16 KB/sec
Laser printer	100 KB/sec
Scanner	400 KB/sec
Classic Ethernet	1.25 MB/sec
USB (Universal Serial Bus)	1.5 MB/sec
Digital camcorder	4 MB/sec
IDE disk	5 MB/sec
40x CD-ROM	6 MB/sec
Fast Ethernet	12.5 MB/sec
ISA bus	16.7 MB/sec
EIDE (ATA-2) disk	16.7 MB/sec
FireWire (IEEE 1394)	50 MB/sec
XGA Monitor	60 MB/sec
SONET OC-12 network	78 MB/sec
SCSI Ultra 2 disk	80 MB/sec
Gigabit Ethernet	125 MB/sec
Ultrium tape	320 MB/sec
PCI bus	528 MB/sec
Sun Gigaplane XB backplane	20 GB/sec

设备管理的任务

1. 按照用户的请求，控制设备的各种操作，完成I/O设备与内存之间的数据交换(包括设备分配与回收、设备驱动程序、设备中断处理、缓冲区管理)，最终完成用户的I/O请求
2. 向用户提供使用外部设备的方便接口，使用户摆脱繁琐的编程负担
3. 充分利用各种技术(通道，中断，缓冲等)提高CPU与设备、设备与设备之间的并行工作能力，充分利用资源，提高资源利用率

设备管理的任务

4. 保证在多道程序环境下，当多个进程竞争使用设备时，按一定策略分配和管理各种设备，使系统能有条不紊的工作
5. 保护：设备传送或管理的数据应该是安全的、不被破坏的、保密的
6. 与设备无关性(设备独立性)
 - 用户在编制程序时，使用逻辑设备名，由系统实现从逻辑设备到物理设备(实际设备)的转换
 - 用户能独立于具体物理设备而方便的使用设备

下述有关设备独立性的描述中，正确的是

- ☐ A 指I/O设备具有独立执行I/O功能的一种特性
- ☒ B 指用户程序独立于具体物理设备的一种特性
- ☐ C 指能够实现设备共享的一种特性
- ☐ D 指设备驱动程序独立于具体物理设备的一种特性

提交

大纲

01

设备管理概述

02

I/O硬件组成原理

03

I/O软件技术

04

I/O软件层次

I/O硬件组成原理

- I/O设备的分类
- 设备的组成
- 端口编址方法
- I/O控制方式

I/O设备的分类

■按数据组织分类

■块设备

- 以数据块为单位存储、传输信息

■字符设备

- 以字符为单位存储、传输信息

■有些设备既不是块设备，也不是字符设备，例如时钟

I/O设备的分类

■按功能特性分类

- 存储型设备
- 输入输出型设备(交互型设备)
- 数据通信设备

I/O设备的分类

■从资源分配角度分类

■独占设备

- 在一段时间内只能有一个进程使用的设备，一般为低速I/O设备(如打印机，磁带等)

■共享设备

- 在一段时间内可有多个进程共同使用的设备，多个进程以交叉的方式来使用设备，其资源利用率高(如硬盘)

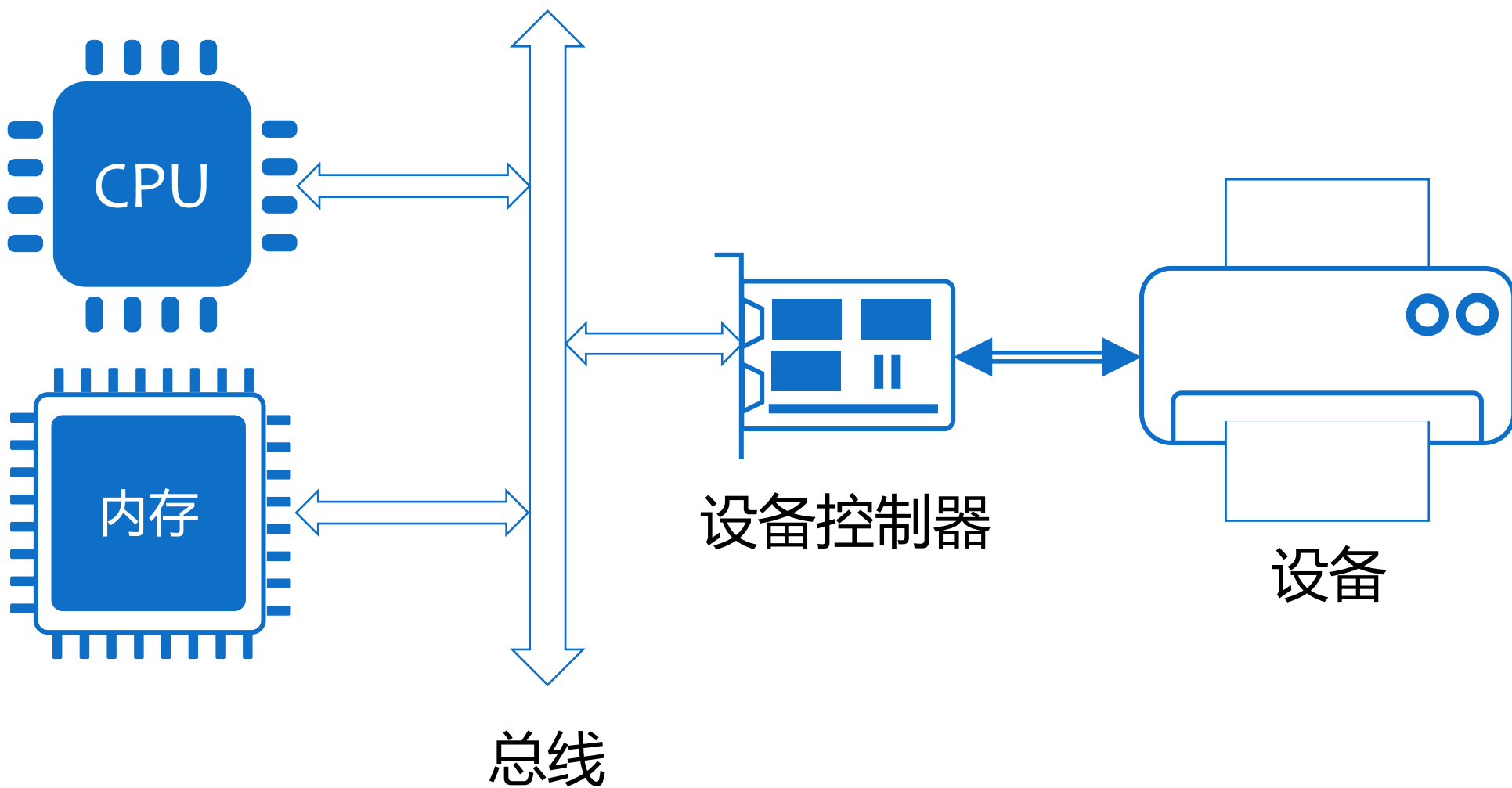
设备的组成

- I/O设备一般由机械和电子两部分组成
 - 机械部分是设备本身(物理装置)
 - 电子部分叫做设备控制器或适配器
- 把这两部分分开处理，以提供更加模块化，更加通用的设计

设备控制器

- 实现设备与主机间的连接和通信
- 设备控制器与主机之间通过标准总线相连接，设备控制器与设备之间的接口一般是私有的
- 通过设备控制器中的若干接口寄存器或接口缓冲区与CPU通信

设备的组成



设备控制器的功能

- 设备端口地址译码
- 按照主机与设备之间约定的格式和过程
 - 接受计算机发来的数据和控制信号
 - 向主机发送数据和状态信号
- 将计算机的数字信号转换成机械部分能识别的模拟信号，或反之
- 实现设备内部硬件缓冲、数据加工等提高性能或增强功能

设备接口

- 每个设备控制器有几个寄存器用来与CPU进行通信——控制寄存器、状态寄存器、数据缓冲区
- 操作系统将命令写入设备控制器的控制寄存器(或数据缓冲区)中，以实现输入/输出，并从状态寄存器(或数据缓冲区)读取状态信息或结果信息
 - 例如：IBM PC的软盘控制器可接收15条命令，READ、WRITE、FORMAT、SEEK、RECALIBRATE，命令可以带参数，它们被一起送入控制器的寄存器中

设备接口

- 当控制器接受一条命令后，可独立于CPU完成指定操作，CPU可以转去执行其它运算
- 命令完成时，控制器产生一个中断，CPU响应中断，控制转给操作系统
- 通过读设备控制器的状态寄存器中的信息，获得操作结果和设备状态

设备接口

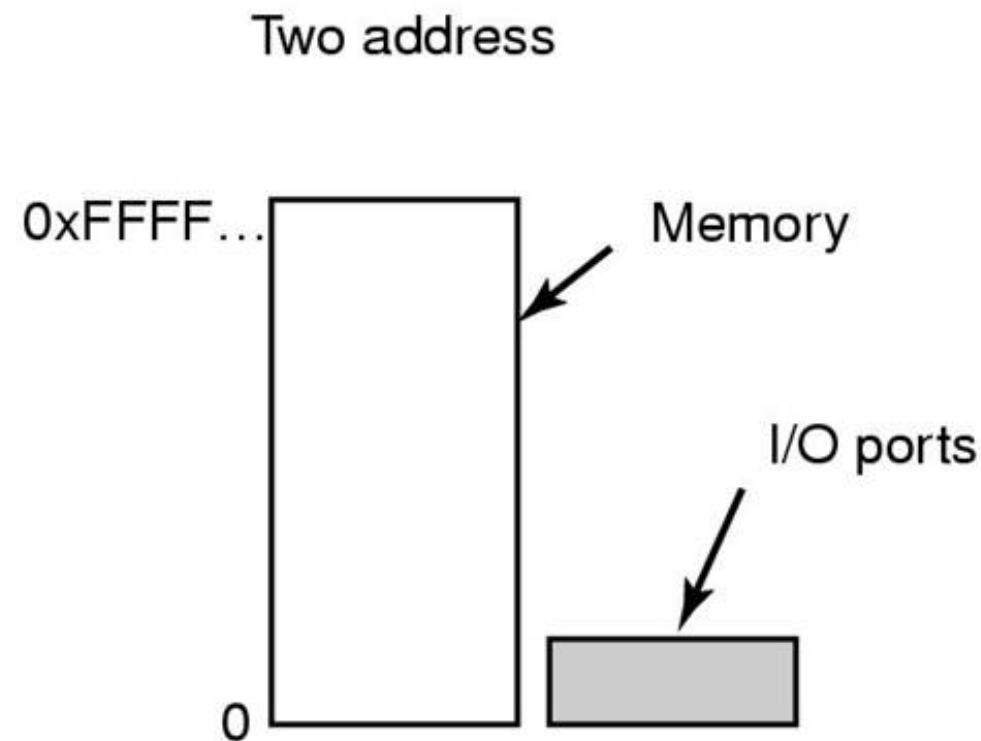
- 控制器与设备之间的接口常常是一个低级接口
 - 例如磁盘，可以按每个磁道8扇区，每个扇区512个字节进行格式化。然而，实际从驱动器出来的却是一连串的位流，以一个前导符(preamble)开始，然后是一个扇区的4096位(512×8)，最后是检查和或错误校验码(ECC)。前导符是在对磁盘格式化时写上的，它包括柱面和扇区数，扇区的大小和类似的一些数据
- 控制器的任务是把串行的位流转换为字节块，并进行必要的错误修正。首先，控制器按位进行组装，然后存入控制器内部的缓冲区中形成以字节为单位的块。在对块验证检查和并证明无错误时，再将它复制到主存中

端口编址方法

- 设备控制器中有多个寄存器(称为I/O端口), 每个寄存器有唯一的一个地址, 该地址称为I/O端口地址。
- I/O端口地址主要有两种形式, 与I/O指令的形式相关联
 - 内存映射编址(内存映射I/O模式)
 - I/O独立编址(I/O专用指令)

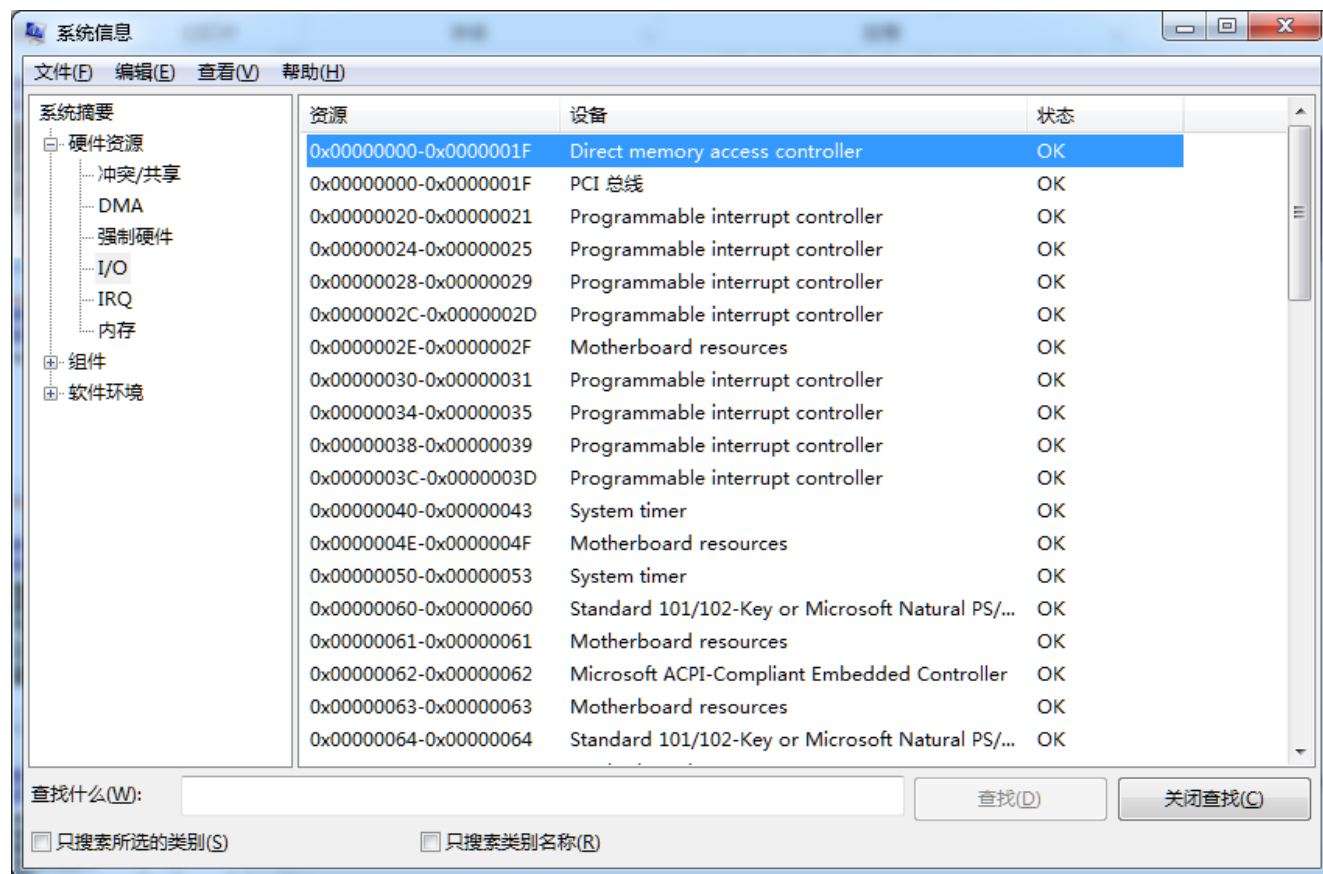
I/O独立编址

- 每个寄存器被分配一个I/O端口号。分配给系统中所有端口的地址空间是完全独立的，与内存的地址空间没有关系
- 主机使用专门的I/O指令对端口进行操作
- 例子
 - 8086/8088，分配给I/O端口的地址空间0000H~0FFFFH，64K，只能用IN和OUT指令对其进行读写操作



I/O独立编址

■ PC机的I/O端口地址



内存映射编址

- 分配给系统中所有端口的地址空间与内存的地址空间统一编址
- 主机把I/O端口看作一个存储单元，对I/O的读写操作等同于对存储器的操作
- 大部分处理器采用内存映射I/O

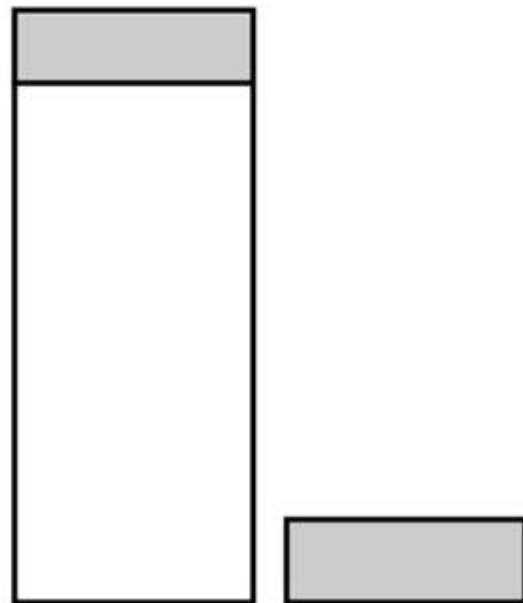
One address space



混合方案

- 具有内存映射I/O的数据缓冲区，而控制/状态寄存器则具有单独的I/O端口
- PC机采用I/O混合编址方案

Two address spaces



内存映射I/O的优点

- C/C++不存在执行IN/OUT指令的方法，所以独立编址I/O访问设备寄存器只能采用汇编代码；而对于内存映射I/O，设备寄存器只是内存中的变量，可以和其它变量一样寻址
- 对于内存映射I/O，不需要特殊的保护机制来阻止用户进程执行I/O。操作系统要做的只是避免将包含设备寄存器的那部分地址映射到任何用户的虚拟地址空间之中

内存映射I/O的优点

- 对于内存映射I/O，可以访问内存的每条指令也可以访问设备寄存器
- 例如，如果存在TEST指令可以测试一个内存字是否为0，则也可用来测试某个设备状态寄存器是否为0

```
LOOP:      TEST      PORT_4
            BEQ       READY
            BRANCH    LOOP

READY:
```

内存映射I/O的缺点

- 高速缓存问题：缓存一个设备寄存器可能是灾难性的

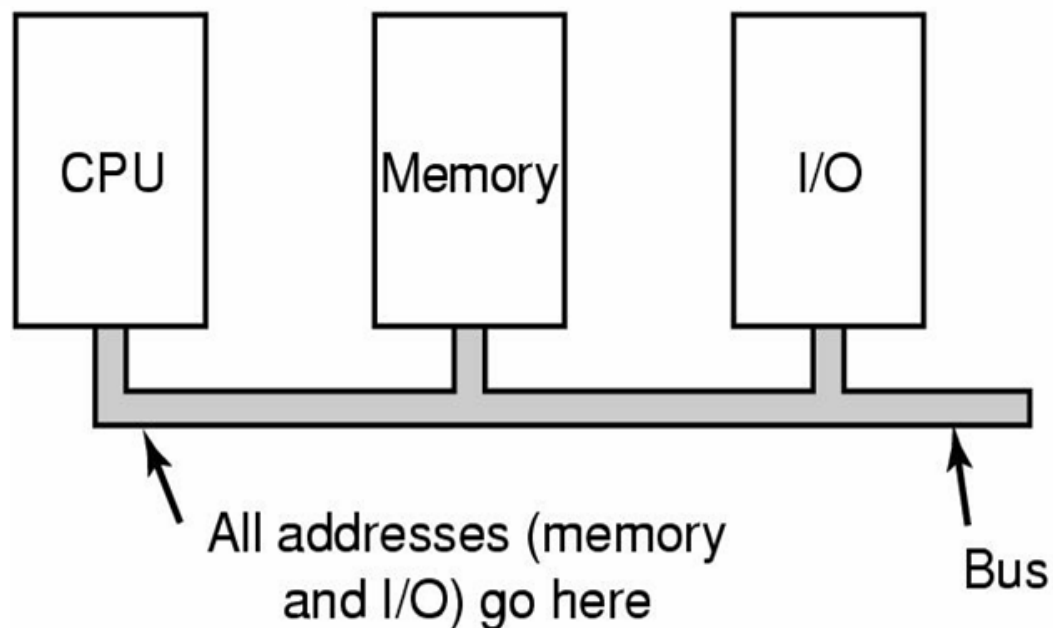
```
LOOP:      TEST      PORT_4
            BEQ       READY
            BRANCH    LOOP

READY:
```

- 为避免这一问题，硬件必须针对每个页面具备选择性禁止高速缓存的能力

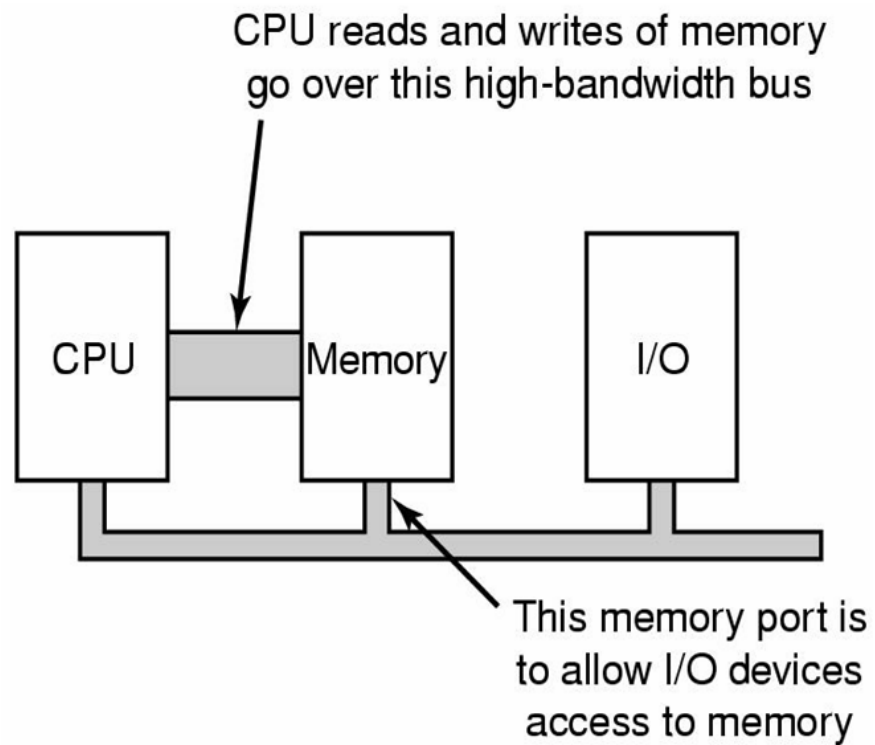
内存映射I/O的缺点

- 只存在单一内存空间，所以所有内存模块和设备都必须检查所有内存访问。采用单总线结构是自然的，但是影响系统性能



内存映射I/O的缺点

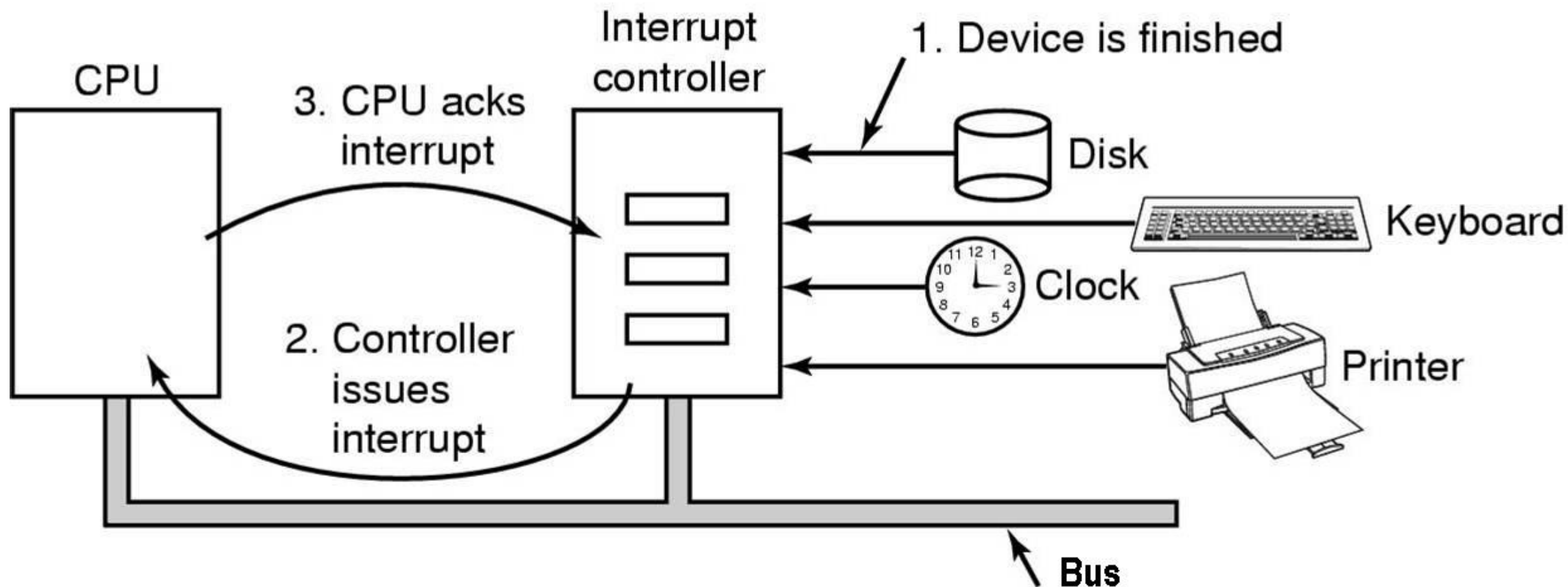
- 为提高性能，现代计算机系统多采用多总线机构。必须采取特殊的措施使内存映射I/O工作在多总线系统上



I/O控制方式

- 程序控制I/O
- 中断驱动I/O
- DMA
- 通道

中断驱动I/O



中断驱动I/O

- 为了使CPU能够处理中断，中断控制器向CPU发出中断信号的同时在地址总线上放置一个数字表明哪个设备需要关注
- 中断信号导致CPU暂停当前工作。地址总线上的数字被用作指向中断向量表的索引
- 中断向量表中存放中断服务程序的入口地址

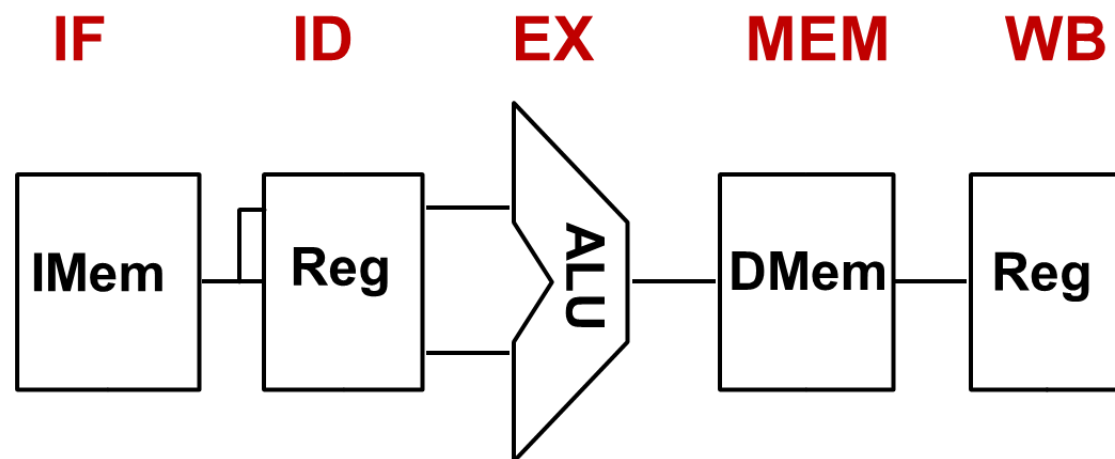
中断驱动I/O

- 在中断服务程序开始前，硬件必须保存一定的信息。最低限度要保存被中断的进程的程序计数器的值，另一个极端是保存所有寄存器的值
- 信息保存的位置
 - CPU内部的寄存器——中断无法嵌套
 - 栈——当前用户进程栈、内核栈

中断驱动I/O

■流水线和超标量对中断处理的影响

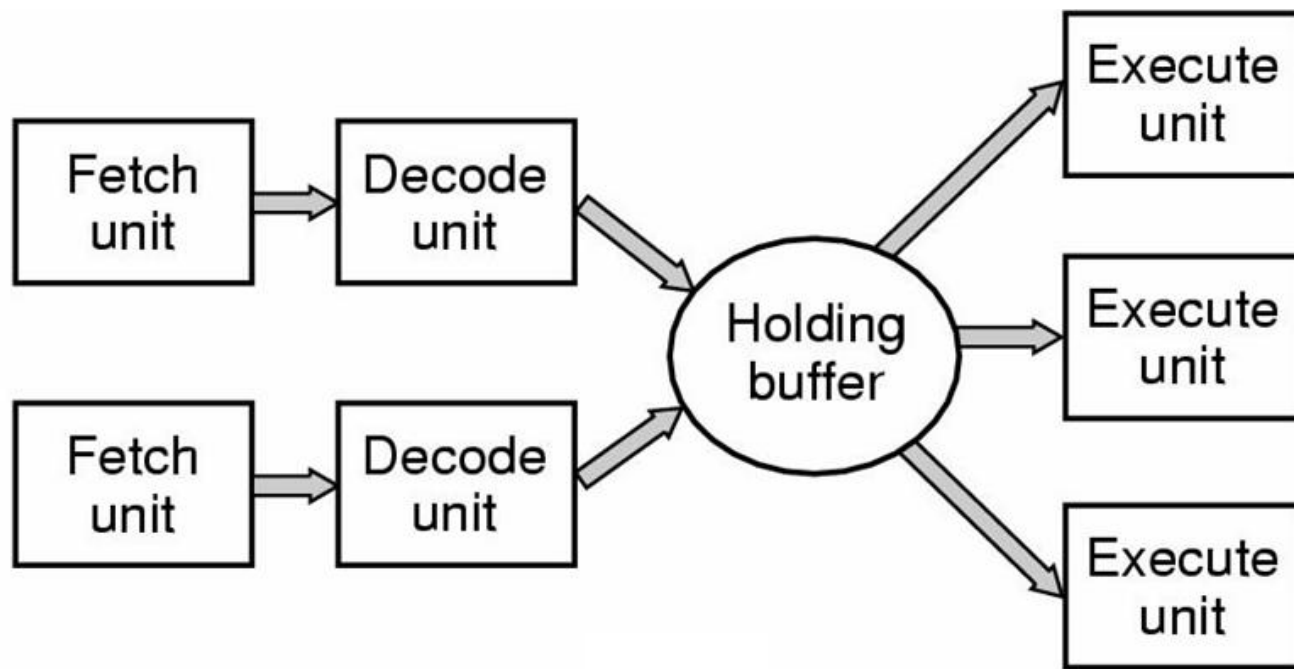
- 在流水线满的时候，当中断出现时，程序计数器的值可能无法正确反映已经执行过的指令和尚未执行的指令之间的边界



中断驱动I/O

■流水线和超标量对中断处理的影响

- 在超标量计算机上，因为指令可能乱序执行，所以在已经执行过的指令和尚未执行的指令之间可能并不存在明确的边界



直接存储器存取DMA

- 数据在内存与I/O设备间的直接成块传送
- CPU在开始时向设备发“传送一块”命令，结束时进行相应处理，实际操作由DMA控制器硬件直接完成
- DMA的功能可以以独立的DMA部件在系统I/O总线上完成，也可整合到I/O部件中完成
- DMA控制器能够通过系统总线代替CPU管理数据的存入或取出
 - 当CPU不需要系统总线时可以使用总线
 - DMA可以强迫CPU暂时延迟其他操作，获取一个总线周期(周期窃取)

直接存储器存取DMA

■ 没有DMA时，磁盘如何读：

- 首先，控制器从磁盘驱动器串行地一位一位地读一个块，直到将整块信息放入控制器的内部缓冲区中
- 其次，它做和校验计算，以核实没有读错误发生
- 然后控制器产生一个中断。CPU响应中断，控制转给操作系统。当操作系统开始运行时，它重复地从控制器缓冲区中一次一个字节或一个字地读这个磁盘块的信息，并将其送入内存中

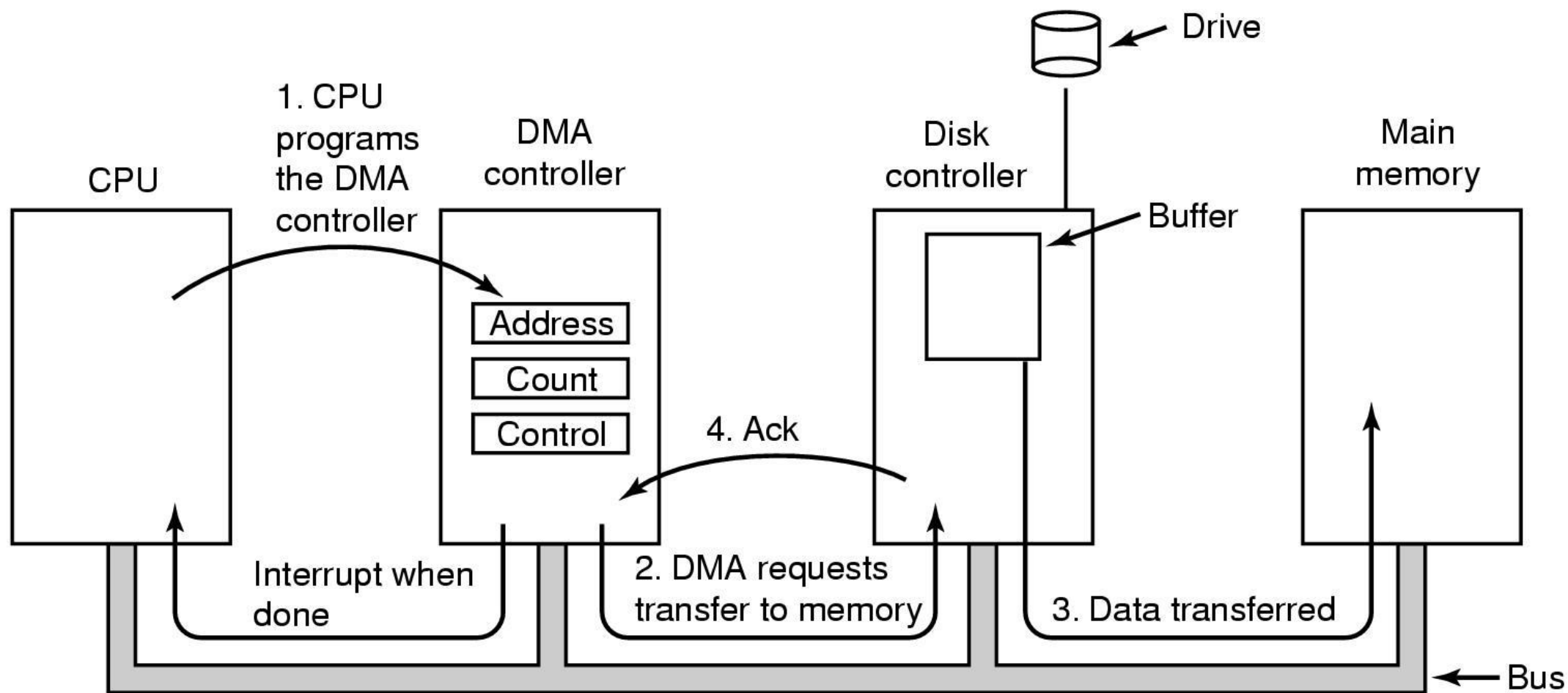
直接存储器存取DMA

- 采用DMA方式时，允许DMA控制器接管地址线的控制权，直接控制DMA控制器与内存的数据交换。从而使磁盘设备与存储器之间的数据传送不需要CPU介入，因而减轻了CPU负担
- 当采用DMA时，除向控制器提供要读块的磁盘地址外，还要向控制器提供两个信息：
 - 要读块送往内存的起始地址
 - 要传送的字节数

直接存储器存取DMA

- 当DMA硬件控制磁盘与存储器之间进行信息交换时，每当磁盘把一个数据读入控制器的数据缓冲区时，DMA控制器取代CPU，接管地址总线的控制权，并按照DMA控制器中的存储器地址寄存器内容把数据送入相应的内存单元中
- 然后，DMA硬件自动地把传送字节计数器减1，把存储器地址寄存器加1，并恢复CPU对内存的控制权，DMA控制器对每一个传送的数据重复上述过程，直到传送字节计数器为“0”时，向CPU产生一个中断信号
- 当操作系统接管CPU控制权时，无需再做块复制的工作

直接存储器存取DMA



DMA方式与中断的主要区别

- 中断方式是在数据缓冲寄存区满后，发中断请求，CPU进行中断处理
- DMA方式则是在所要求传送的数据块全部传送结束时要求CPU进行中断处理
- DMA方式大大减少了CPU进行中断处理的次数
- 中断方式的数据传送是由CPU控制完成的
- 而DMA方式则是在DMA控制器的控制下不经过CPU控制完成的

通道方式

- **通道：**又称为输入输出处理器、I/O处理机
- **通道功能：**能完成主存和外围设备间的信息传送，与CPU并行地执行操作
- **通道的工作特点：**实现外围设备和CPU能并行操作；通道和通道之间能并行操作；各通道上的外围设备并行操作，达到提高整个系统效率这一根本目的
- **评价：**通道技术解决了I/O操作的独立性和各部件工作的并行性。把CPU从琐碎的I/O操作中解放出来。是计算机技术的一个重要进步

通道方式

■采用通道后的I/O操作过程

- 启动I/O操作：CPU在执行主程序时遇到I/O请求，它启动指定通道上选址的外围设备，一旦启动成功，通道开始控制外围设备进行操作。之后CPU就可执行其他任务并与通道并行工作
- 结束I/O操作：I/O通道I/O操作完成后，发出操作结束中断，CPU才停止当前工作，转向处理I/O操作结束事件

通道方式

- 按照信息交换方式和连接设备不同，通道可以分为三种类型
 - 字节多路通道：通道同时为多个低速I/O设备(软盘、纸带、磁带机等)服务
 - 选择通道：通道分时地为各个高速I/O设备(磁盘、磁带机等)服务
 - 数组多路通道：通道同时地为多个像磁盘机(移臂时间长)一样的I/O设备服务

通道方式

- 按照信息交换方式和连接设备不同，通道可以分为三种类型
 - 字节多路通道：通道同时为多个低速I/O设备(软盘、纸带、磁带机等)服务
 - 选择通道：通道分时地为各个高速I/O设备(磁盘、磁带机等)服务
 - 数组多路通道：通道同时地为多个像磁盘机(移臂时间长)一样的I/O设备服务

按___分类可将设备分为块设备和字符设备

- ☐ A 从属关系
- ☐ B 操作特性
- ☐ C 共享属性
- ☒ D 信息交换单位

提交

I/O设备与内存之间的数据交换不经过CPU来完成，这种I/O方式为

- ☐ A 程序查询方式
- ☐ B 中断方式
- ☒ C DMA方式
- ☐ D 外部总线方式

提交

大纲

01

设备管理概述

02

I/O硬件组成原理

03

I/O软件技术

04

I/O软件层次

设备独立性

- 使用户或程序员可以访问任何I/O设备而无需事先指定设备
- 例如，命令

```
sort <input >output
```

- 无论输入来自软盘、硬盘或者键盘，输出送往任意类型的磁盘或屏幕，都可以正常工作。尽管这些设备的差别很大，其处理由操作系统负责

统一命名

- 对输入输出设备采用与文件统一的命名方式，所有文件和设备都使用相同的方式：通过路径名进行寻址
- 例如，在UNIX系统中，像/dev/tty00这样的文件名，代表一个设备

错误处理

- 错误应该在尽可能靠近硬件的层面得到处理。在很多情况下，错误处理可以在低层透明地得到解决，高层软件甚至不知道存在这一错误
- 大多数错误是与设备密切相关的，因此，只有驱动程序知道应如何处理(比如，重试、忽略或放弃)。但还有一些典型的错误不是输入输出设备的错误造成的，如由于磁盘块受损而不能再读，驱动程序将尝试重读一定次数，若仍有错误，则放弃重读并通知与上层与设备无关的软件

同步和异步传输

- 同步传输为阻塞式传输，异步传输为非阻塞式(中断驱动)传输
- 大多数物理I/O是异步的，但是阻塞式I/O使用户程序更加容易编写
- 操作系统使实际上是中断驱动的I/O变为在用户看来是阻塞式的操作

缓冲技术

- CPU和I/O设备的速度差异是明显的，CPU的速度以纳秒计，外部设备的处理速度则一般以毫秒甚至秒计。虽然操作系统为各部分并行工作提供了可能，但是由于CPU和外部设备速度不匹配，它们的并行程度并不能得到充分发挥
- 必须引入缓冲技术

缓冲技术

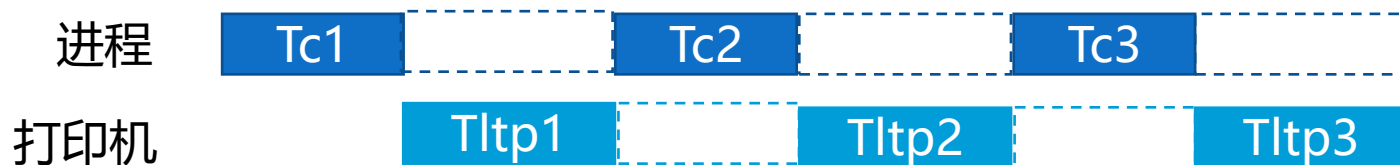
■例：假设在一段时间内，系统中只有一个用户进程正在使用行式打印机。其工作过程是：

- ① 计算并产生一行需打印的信息，它们存放在字符型数组 `line[LINELENGTH]` 中，所需时间用 T_c 表示
- ② 要求系统将 `line` 数组中的信息送行式打印机打印，所需时间表示为 T_{lpt}

缓冲技术

■无缓冲

- 为了使用行式打印机，处理机调度操作比较频繁，因此也就增加了系统开销，同时，打印机的忙闲程度也很不均匀，不能充分实施并行操作



缓冲技术

■单缓冲

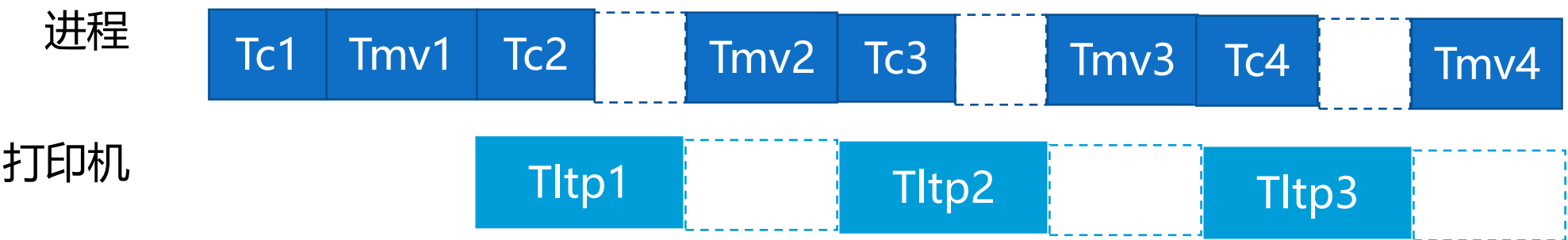
- 在系统中为行式打印机设置一个缓冲存储区buffer[LINELENGTH]

- 于是，进程使用行式打印机的过程变成：

- ① 计算并产生一行需打印的信息，它们存放在line[LINELENGTH]中
- ② 要求系统将line数组中的信息交行式打印机打印
- ③ 系统将line数组中的信息先传送到buffer中，这一工作所需时间记为Tmv，然后命令行式打印机将buffer中的内容打印出来

缓冲技术

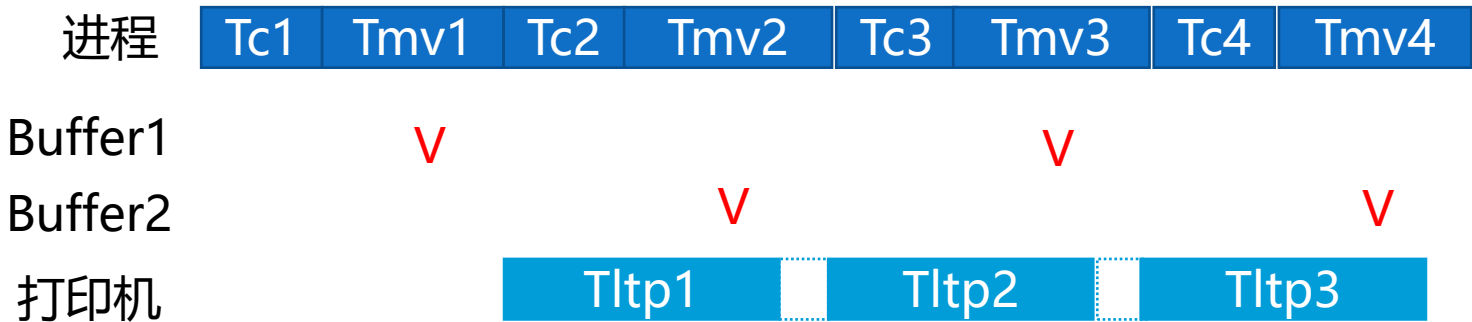
■单缓冲



缓冲技术

■多缓冲

- 为了进一步提高行式打印机的利用程度并减少进程调度次数，可以继续增加缓存数



共享设备与独占设备

- 独占设备每次只能分配给一个进程使用，这种使用特性隐含着死锁的必要条件，所以在考虑独占设备的分配时，一定要结合有关防止和避免死锁的安全算法
- 共享设备是可由若干个进程同时共享的设备，例如磁盘机

共享设备与独占设备

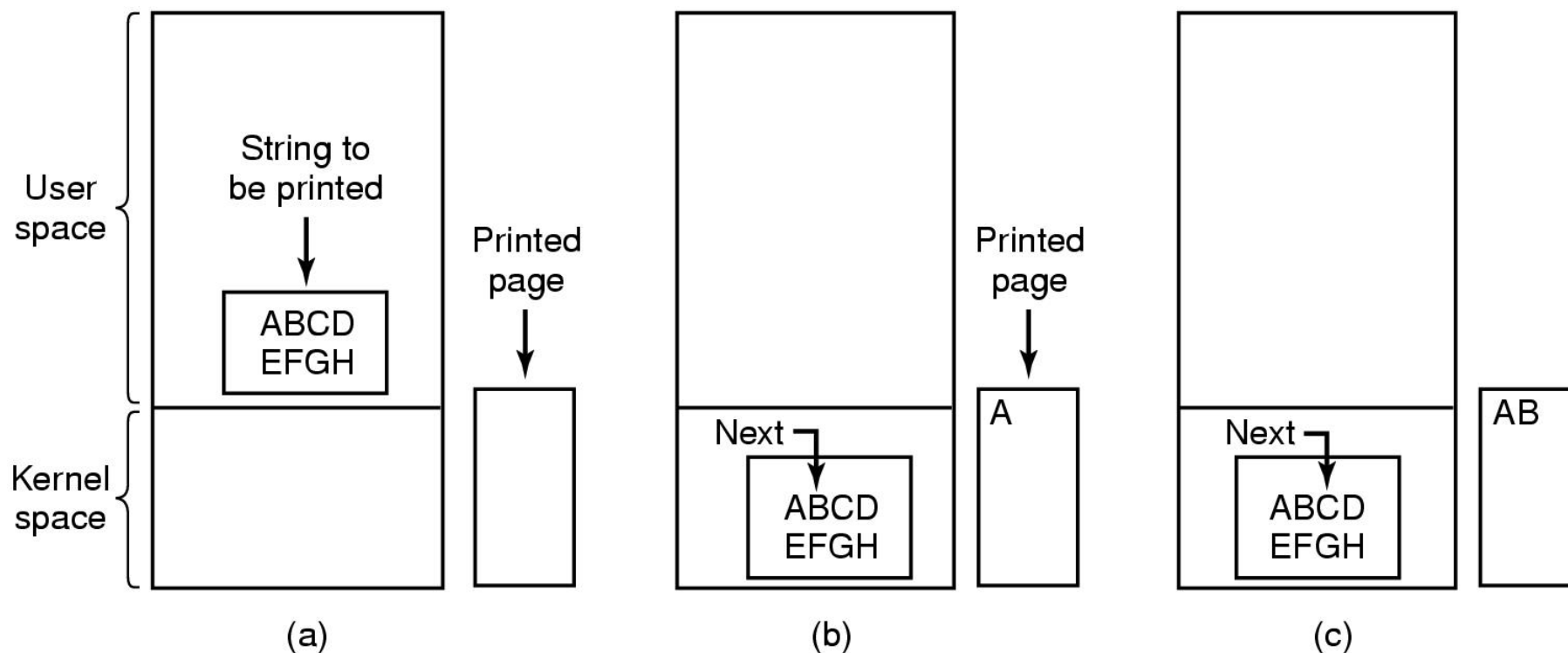
- 系统中的独占设备是有限的，往往不能满足诸多进程的要求，因而会引起大量进程由于等待某些独占设备而阻塞，成为系统中的“瓶颈”。另一方面，申请到独占设备的进程在其整个运行期间虽然占有设备，利用率却常常很低，设备还是经常处于空闲状态
- 为了解决这种矛盾，最常用的方法就是用共享设备来模拟独占设备的操作，从而提高系统效率和设备利用率。实现这一技术的软、硬件系统被称为SPOOLing系统(Simultaneous Peripheral Operation On Line, 同时外围联机操作)

共享设备与独占设备

- 例如，打印机是一种典型的独占设备，引入SPOOLing技术后，用户的打印请求传递给SPOOLing系统，而并不是真正把打印机分配给用户
- SPOOLing系统的输出进程在磁盘上申请一个空闲区，把需要打印的数据传送到里面，再把用户的打印请求挂到打印队列上。如果打印机空闲，就会从打印队列中取出一个请求，再从磁盘上的指定区域取出数据，执行打印操作
- 由于磁盘是共享的，SPOOLing系统可以随时响应打印请求并把数据缓存起来，这样就把独占设备改造成了共享设备，从而提高了设备的利用率和系统效率

程序控制I/O

- 程序控制I/O(PIO, Programmed I/O), 也称为直接I/O或轮询
- 由CPU负责在机器内存与设备控制器数据寄存器之间进行数据传送



程序控制I/O

- 缺点：直到全部I/O完成之前要占用CPU的全部时间

```
copy_from_user(buffer, p, count);  
for (i = 0; i < count; i++) {  
    while (*printer_status_reg != READY) ;  
    *printer_data_register = p[i];  
}  
return_to_user( );
```

/* p is the kernel bufer */
/* loop on every character */
/* loop until ready */
/* output one character */

中断驱动I/O

■减少程序控制I/O中CPU的等待时间，提高系统的并行工作程度

```
copy_from_user(buffer, p, count);  
enable_interrupts( );  
while (*printer_status_reg != READY) ;  
*printer_data_register = p[0];  
scheduler( );
```

(a)

```
if (count == 0) {  
    unblock_user( );  
} else {  
    *printer_data_register = p[i];  
    count = count - 1;  
    i = i + 1;  
}  
acknowledge_interrupt( );  
return_from_interrupt( );
```

(b)

■缺点：中断发生在每个字符上，浪费一定数量的CPU时间

使用DMA的I/O

- 将中断的次数从打印每个字符一次减少到每个缓冲区一次

```
copy_from_user(buffer, p, count);  
set_up_DMA_controller( );  
scheduler( );
```

(a)

```
acknowledge_interrupt( );  
unblock_user( );  
return_from_interrupt( );
```

(b)

在SPOOLing系统中，用户进程实际分配到的的是——。

- ☐ A 用户所请求的外设
- ☒ B 内存区、虚拟设备
- ☐ C 设备的缓冲区
- ☐ D 设备的一部分空间

提交

若I/O操作所花的时间比CPU处理的时间短很多，
则缓冲区_____。

- ☐ A 有必要
- ☒ B 无必要
- ☐ C 可能有必要，可能无必要

大纲

01

设备管理概述

02

I/O硬件组成原理

03

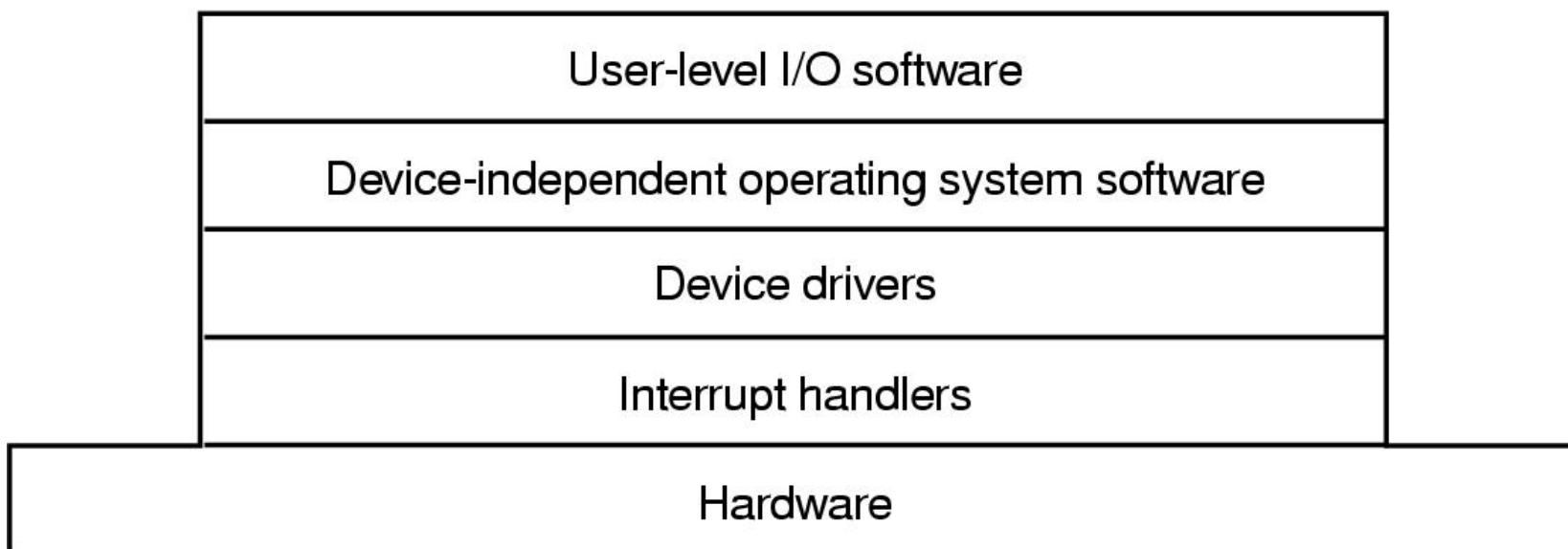
I/O软件技术

04

I/O软件层次

I/O软件层次

- I/O软件的基本思想是按分层的思想构成，较低层软件要使较高层软件独立于硬件的特性，较高层软件则要向用户提供一个友好的、清晰的、简单的、功能更强的接口



I/O软件各层的功能

- **用户进程层**执行输入输出系统调用，对I/O数据进行格式化，为假脱机输入/输出作准备
- **与设备无关的I/O软件**实现设备的命名、设备的保护、成块处理、缓冲技术和设备分配
- **设备驱动程序**设置设备寄存器、检查设备的执行状态
- **中断处理程序**负责I/O完成时，唤醒设备驱动程序进程，进行中断处理
- **硬件层**实现物理I/O的操作

中断处理程序

- 中断应该深深隐藏在操作系统内部
- 隐藏的办法是每个进程在启动一个I/O操作后阻塞，直到I/O操作完成并产生一个中断由操作系统接管CPU后唤醒该进程为止

中断处理程序

- 驱动程序阻塞自己的手段
 - 在信号量上执行P操作
 - 在条件变量上执行wait操作
 - 在消息上执行receive操作
- 中断发生时，中断处理程序将对中断进行处理，然后将启动被阻塞的驱动程序解除阻塞
 - 在信号量上执行V操作
 - 在条件变量上执行signal操作
 - 在消息上执行send操作

设备驱动程序

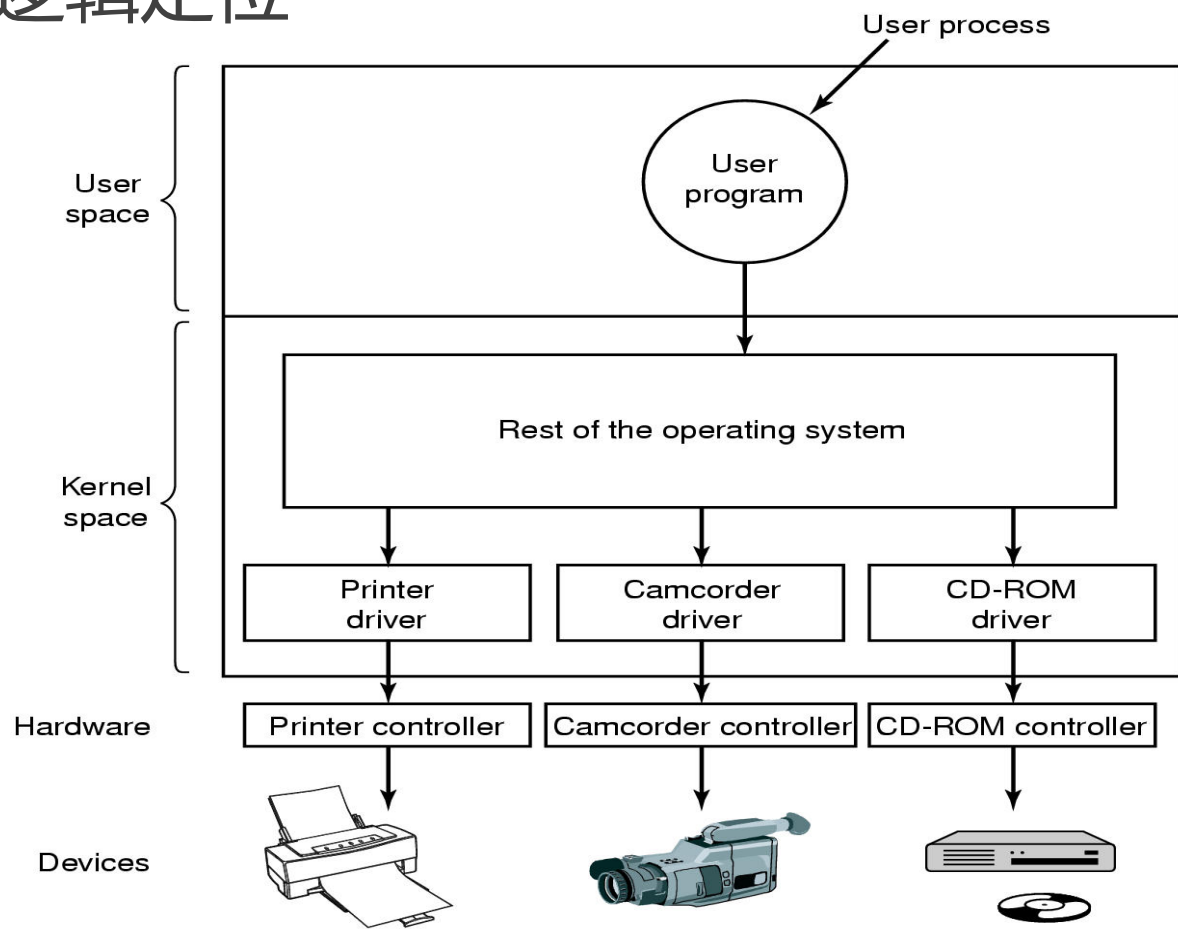
- 与设备密切相关的代码放在设备驱动程序中，每个设备驱动程序处理一种设备类型
- 每一个控制器都设有一个或多个设备寄存器，用来存放向设备发送的命令和参数。设备驱动程序负责释放这些命令，并监督它们正确执行

设备驱动程序

- 一般而言，设备驱动程序的任务是接收来自与设备无关的上层软件的抽象请求，并执行这个请求
- 在设备驱动程序的进程发出一条或多条命令后，系统有两种处理方式
- 多数情况下，执行设备驱动程序的进程必须等待命令完成，这样，在命令开始执行后，它阻塞自己，直到中断处理时将它解除阻塞为止
- 另一些情况下，命令执行不必延迟就很快完成

设备驱动程序

■ 设备驱动程序的逻辑定位



与设备无关的I/O软件

- 虽然I/O软件中一部分是设备特定的，但大部分软件是与设备无关的
- 设备驱动程序和与设备无关的软件之间的确切界限依赖于具体系统

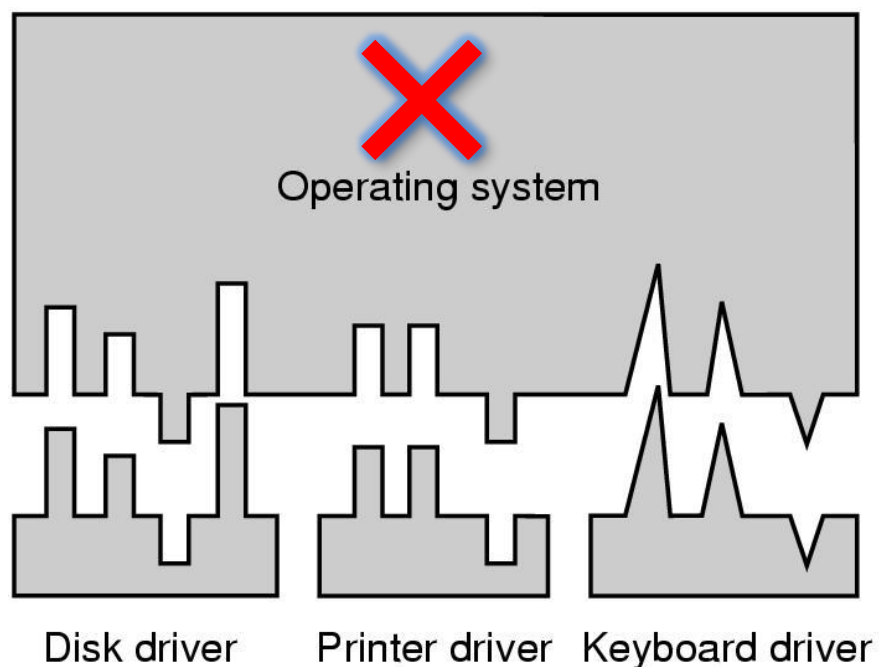
与设备无关的I/O软件

■与设备无关的I/O软件的功能：

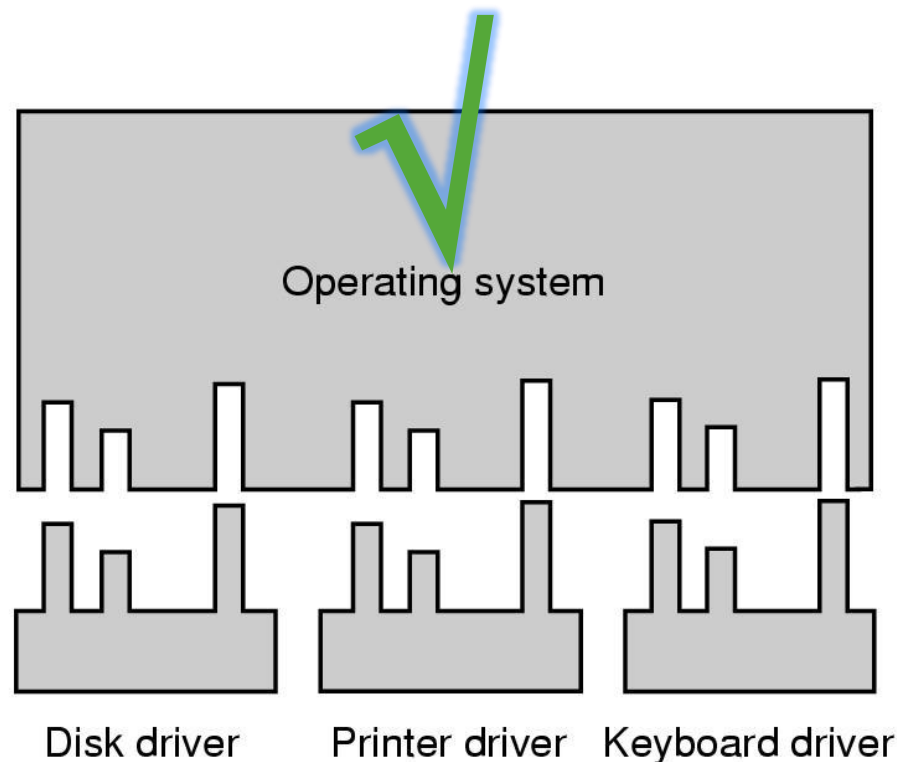
- 设备驱动程序的统一接口
- 设备命名
- 缓冲
- 错误报告
- 分配与释放专用设备
- 提供与设备无关的块大小

与设备无关的I/O软件

■设备驱动程序的统一接口



(a)



(b)

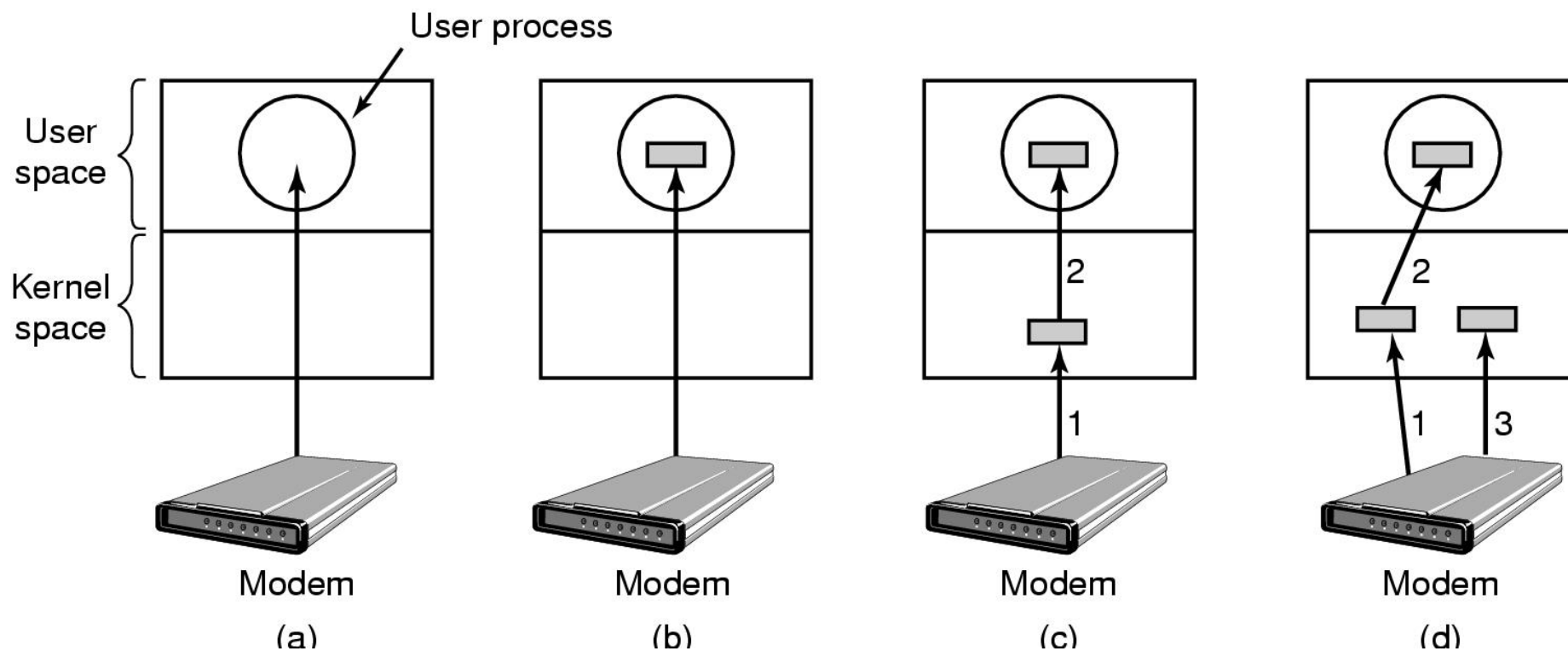
与设备无关的I/O软件

■设备命名

- 对输入输出设备采用与文件统一的命名方式
- 区分这些命名同文件一样的输入输出设备的是与设备无关的软件，它负责把设备的符号名映射到相应的设备驱动程序上
 - 例如，在UNIX系统中，像/dev/disk0这样的文件名，唯一确定了一个特殊文件的i节点，这个i节点包含了主设备号和次设备号。主设备号用于寻找对应的设备驱动程序，而次设备号提供了设备驱动程序的有关参数，用来确定要读写的具体设备

与设备无关的I/O软件

■ 缓冲



与设备无关的I/O软件

■ 错误报告

- 许多错误是设备特定的，必须由驱动程序来处理，但是错误处理的框架应该是与设备无关的
- 编程错误——返回错误代码给调用者
- 实际I/O错误——由驱动程序解决，如果驱动程序解决不了问题，则将问题向上传递给与设备无关的I/O软件。显示对话框、杀死进程...

与设备无关的I/O软件

■ 分配与释放专用设备

■ 分配方法如下：

- ① 根据申请的设备类型查询系统设备表，找到对应入口
- ② $P(S_m)$
- ③ 查对应的设备表，找一空闲设备并分配

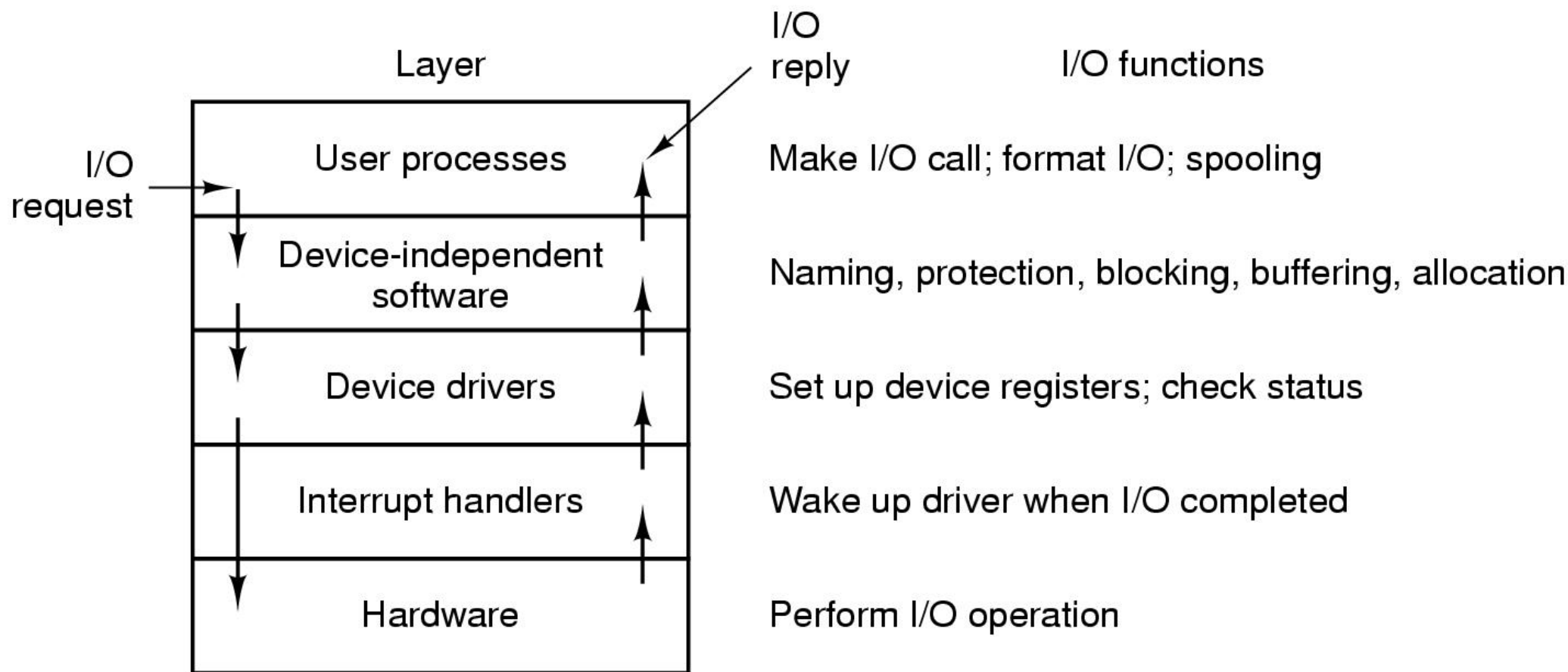
■ 释放方法如下：

- ① 根据释放设备类型查询系统设备表，找到对应入口
- ② 查对应的设备表，找到要释放的设备去配
- ③ $V(S_m)$

用户空间的I/O软件

- 尽管大部分I/O软件都包含在操作系统中，但仍有一小部分是由与用户程序连接在一起的库过程，甚至完全由运行于内核外的程序构成
- 系统调用，包括I/O系统调用，通常由库过程实现
- 这些过程所做的工作只是将系统调用时所用的参数放在合适的位置，由其它的I/O过程实际实现真正的操作

I/O软件层次



将命令写到设备寄存器是在四个I/O软件层的哪一层完成的？

- ☐ A 用户级I/O软件
- ☐ B 与设备无关的I/O软件
- ☒ C 设备驱动程序
- ☐ D 中断处理程序

提交

检查用户是否允许使用设备是在四个I/O软件层的哪一层完成的?

- ☐ A 用户级I/O软件
- ☒ B 与设备无关的I/O软件
- ☐ C 设备驱动程序
- ☐ D 中断处理程序

提交

操作系统，2026

谢谢

OPERATING SYSTEM